API Client Design & Resilient Error Handling

A Practical Guide to Writing Production SDK Wrappers in Python

Reference Resource: Python Cookbook (Ch 14) & Google API Client Architecture | Week 1

Table of Contents

- **Section 1: The Client Adapter Pattern**
  Why we wrap third-party SDKs and avoid leaky abstractions
- **Section 2: Custom Exception Hierarchies**
  Designing domain-specific exceptions for calendar errors
- **Section 3: Exception Chaining (raise ... from)**
  Preserving root-cause stack traces without leaking raw API types
- **Section 4: Handling Google API HttpError**
  Status codes (401, 403, 404, 503) and payload parsing
- **Section 5: Token Expiration & Safe Refresh**
  Handling OAuth2 lifecycle and token file corruption
- **Section 6: Complete Calendar Client Template**
  A production-grade, typed client ready for implementation

Primary Goal for Step 2: Wrap raw Google Calendar API calls inside a robust class that exposes clean Python methods, never leaks raw Google dictionaries, and translates network/API failures into actionable domain exceptions.

1. The Client Adapter Pattern

In AI engineering, an LLM agent needs deterministic tools. If your agent calls the Google Calendar API directly using raw scripts (like the original API.py), low-level errors (network drops, expired tokens, schema changes) leak straight into the agent loop and cause hallucinated recoveries.

The **Client Adapter Pattern** isolates external dependencies behind a clean interface: your scheduler domain only speaks in typed CalendarEvent objects, while GoogleCalendarClient handles authentication, request payloads, and API status codes.

2. Custom Exception Hierarchies

Never catch generic Exception and return None. Design a clear hierarchy inheriting from a base domain exception:

```
# src/core/exceptions.py

class CalendarError(Exception):
    """Base exception for all calendar operations."""
    pass

class CalendarAuthError(CalendarError):
    """Raised when OAuth tokens are missing, expired, or invalid."""
    pass

class CalendarNetworkError(CalendarError):
    """Raised on network timeouts or connection drops."""
    pass

class CalendarResourceNotFoundError(CalendarError):
    """Raised when an event ID does not exist (HTTP 404)."""
    pass

class CalendarRateLimitError(CalendarError):
    """Raised when Google API quota is exceeded (HTTP 403/429)."""
    pass
```

3. Exception Chaining (raise ... from)

When catching a low-level library exception (like googleapiclient.errors.HttpError), always re-raise your custom domain exception using the from keyword. This preserves the original traceback for debugging while presenting a clean interface to callers:

```
from googleapiclient.errors import HttpError

try:
    response = service.events().delete(calendarId="primary", eventId=event_id).execute()
except HttpError as err:
    if err.resp.status == 404:
        raise CalendarResourceNotFoundError(f"Event {event_id} does not exist.") from err
    raise CalendarError(f"Failed to delete event {event_id}: {err}") from err
```

4. Handling Google API HttpError Status Codes

Google's API client raises googleapiclient.errors.HttpError on non-2xx HTTP responses. Inspect err.resp.status to determine the appropriate response:

- **401 Unauthorized:** The OAuth access token has expired or revoked. Trigger automatic refresh.
- **403 Quota Exceeded / Rate Limit:** Back off exponentially and retry.
- **404 Not Found:** The requested event or calendar ID does not exist.
- **500 / 503 Backend Error:** Transient Google server failure; safe to retry with backoff.

5. Token Expiration & Safe Refresh

Google OAuth access tokens expire after 3600 seconds (1 hour). Your client must verify validity before every batch of calls and refresh silently using the stored refresh token:

```
import os
from google.auth.transport.requests import Request
from google.oauth2.credentials import Credentials
from google_auth_oauthlib.flow import InstalledAppFlow
from google.auth.exceptions import RefreshError

def get_credentials(token_path="token.json", creds_path="credentials.json"):
    creds = None
    if os.path.exists(token_path):
        creds = Credentials.from_authorized_user_file(token_path, SCOPES)

    if not creds or not creds.valid:
        if creds and creds.expired and creds.refresh_token:
            try:
                creds.refresh(Request())
            except RefreshError as err:
                raise CalendarAuthError("Saved token expired and refresh failed. Re-run OAuth.") from err
        else:
            if not os.path.exists(creds_path):
                raise CalendarAuthError(f"Missing OAuth credentials at {creds_path}")
            flow = InstalledAppFlow.from_client_secrets_file(creds_path, SCOPES)
            creds = flow.run_local_server(port=0)

        with open(token_path, "w") as token:
            token.write(creds.to_json())
    return creds
```

6. Architecture Checklist for src/tools/calendar_client.py

When writing your modernized calendar client class:

1. **Initialize service once:** Store self.service = build('calendar', 'v3', credentials=creds) in __init__.
2. **Input/Output Type Safety:** Methods must accept Python datetime and return CalendarEvent Pydantic instances.
3. **Wrap all API calls:** Every .execute() must be wrapped in a try / except HttpError block.
4. **Idempotency:** Read methods (list_events) can be retried safely; creation calls should check for duplicates before writing.